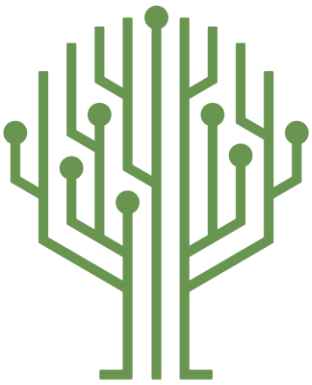


Green Pace Secure Development Policy

Contents

Overview 2
Purpose 2
Scope 2
Module Three Milestone 2
Ten Core Security Principles 2
C/C++ Ten Coding Standards 3
Coding Standard 1 4
Coding Standard 2 5
Coding Standard 3 6
Coding Standard 4 7
Coding Standard 5 8
Coding Standard 6 9
Coding Standard 7 10
Coding Standard 8 11
Coding Standard 9 13
Coding Standard 10 14
Defense-in-Depth Illustration 15
Project One 15
1. Revise the C/C++ Standards 15
2. Risk Assessment 15
3. Automated Detection 15
4. Automation 15
5. Summary of Risk Assessments 16
6. Create Policies for Encryption and Triple A 16
7. Map the Principles 17
Audit Controls and Management 18
Enforcement 18
Exceptions Process 18
Distribution 19
Policy Change Control 19
Policy Version History 19
Appendix A Lookups 19
Approved C/C++ Language Acronyms 19

Overview

Software development at Green Pace requires consistent implementation of secure principles to all developed applications. Consistent approaches and methodologies must be maintained through all policies that are uniformly defined, implemented, governed, and maintained over time.

Purpose

This policy defines the core security principles; C/C++ coding standards; authorization, authentication, and auditing standards; and data encryption standards. This article explains the differences between policy, standards, principles, and practices (guidelines and procedure): Understanding the Hierarchy of Principles, Policies, Standards, Procedures, and Guidelines.

Scope

This document applies to all staff that create, deploy, or support custom software at Green Pace.

Module Three Milestone

Ten Core Security Principles

Principles	Write a short paragraph explaining each of the 10 principles of security.
1. Validate Input Data	Validate Input Data means that any data coming into the system from users, files, or other services is treated as untrusted until it is checked. Developers should verify type, length, format, and allowed ranges before using the data in queries, calculations, or system calls. Doing this helps prevent attacks like SQL injection and buffer overflows by rejecting bad or unexpected input early.
2. Heed Compiler Warnings	Heed Compiler Warnings means paying attention to and fixing warnings that the compiler reports instead of ignoring them. Many warnings point to real bugs such as uninitialized variables, type mismatches, or possible data loss. Treating warnings like errors helps catch security and reliability issues early in development instead of after the code is deployed.
3. Architect and Design for Security Policies	Architect and Design for Security Policies means planning security into the system from the beginning instead of adding it at the end. The architecture should reflect the organization's security policies, such as how data is protected, who can access which features, and how sensitive operations are separated. When design decisions follow clear security policies, it is easier to enforce consistent protections and harder for attackers to find weak spots.
4. Keep It Simple	Keep It Simple means designing code and systems that are as straightforward as possible while still meeting requirements. Complex solutions are harder to understand, test, and maintain, which increases the chance of hidden bugs and security holes. Simple, clear designs make it easier for developers and reviewers to spot mistakes and prove that the code behaves as expected.
5. Default Deny	Default Deny means that the system should block access by default and only allow

Principles	Write a short paragraph explaining each of the 10 principles of security.
	actions that are explicitly permitted. Instead of trying to list everything that is not allowed, developers define clear rules for what is allowed and deny all other requests. This reduces the chance that a forgotten case or new feature accidentally exposes sensitive data or functionality.
6. Adhere to the Principle of Least Privilege	The Principle of Least Privilege means giving each user, process, and component only the minimum access it needs to do its job, and nothing more. For example, a service that only reads data from a database should not also have permission to drop tables or create new admin accounts. Limiting privileges in this way reduces the damage that can be done if an account is compromised or a bug is exploited.
7. Sanitize Data Sent to Other Systems	Sanitize Data Sent to Other Systems means cleaning and validating any data before sending it to databases, web services, logs, or other applications. This includes removing or encoding dangerous characters, enforcing expected formats, and avoiding direct insertion of user data into commands or queries. Proper sanitization helps prevent attacks such as SQL injection, command injection, and log forging when data crosses system boundaries.
8. Practice Defense in Depth	Defense in Depth means using multiple layers of security controls so that if one layer fails, others are still in place to protect the system. For example, an application might use input validation, parameterized queries, authentication, authorization checks, and network firewalls together. Layered defenses make it much harder for an attacker to succeed with a single mistake or vulnerability.
9. Use Effective Quality Assurance Techniques	Use Effective Quality Assurance Techniques means relying on testing, code review, static analysis tools, and other QA practices to find defects early. Security issues often start as basic logic errors, memory problems, or edge cases that good QA can catch before release. Making QA a regular part of development improves both reliability and security of the code base.
10. Adopt a Secure Coding Standard	Adopt a Secure Coding Standard means agreeing on a clear set of coding rules for the team and following them consistently across all projects. These rules cover things like naming, memory management, error handling, and how to avoid common security weaknesses in the chosen language. When everyone uses the same secure standard, it becomes easier to review code, spot problems, and maintain a strong overall security posture over time.

C/C++ Ten Coding Standards

Coding Standard 1

Coding Standard	Label	Name of Standard Use appropriate and explicit data types for security-sensitive values.
Data Type	STD-001-CPP	Choosing the correct data type for each variable helps prevent overflow, truncation, and sign errors. Security-sensitive values such as sizes, counts, and external data should use explicit, fixed-width integer types or types that clearly match their intended range. Clear and appropriate types make the code easier to read and reduce subtle bugs that attackers could exploit.

Noncompliant Code

This example stores a byte count in a signed int and mixes it with an unsigned value. If a large value is passed in or a negative value appears, the implicit conversions can cause incorrect size checks and lead to buffer overflows or logic errors.

```
void copy_data(char* dest, const char* src, int length)
{
     // length comes from an untrusted source
    if (length < 1024)
    {
        // implicit conversions between signed and unsigned
        for (unsigned int i = 0; i < length; ++i)
        {
            dest[i] = src[i];
        }
    }
}
```

Compliant Code

This example uses an explicit unsigned size type for the length and validates it against both the destination buffer size and a maximum allowed value. Using a clear, appropriate type and proper bounds checking makes the copy logic safer and easier to understand.

```
#include <cstddef>
void copy_data_safe(char* dest, std::size_t destSize, const char* src,
std::size_t length)
{
   const std::size_t MAX_COPY = 1024;

   if (length <= destSize && length <= MAX_COPY)
```

Compliant Code

```
    {
        for (std::size_t i = 0; i < length; ++i)
        {
            dest[i] = src[i];
        }
    }
}
```

Principles(s): Principle 1: Validate Input Data. This principle supports the standard by requiring that all external values use types that match their expected range, which reduces overflow and truncation bugs.
Principle 5: Default Deny Access. Using strict, explicit types defaults to safer behavior and avoids permissive implicit conversions unless values are intentionally checked.
Principle 10: Adopt a Secure Coding Standard. This standard directly applies secure coding guidance for integer and size types so that security risks from incorrect types are identified and prevented.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	High	2

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Integer and signed/unsigned conversion checks	Detects unsafe integer usage, such as mixing signed and unsigned types or using potentially overflowing values in size and index calculations.
SonarLint for Visual Studio (C++)	Current classroom version	Integer overflow and SEI CERT-aligned integer rules	Highlights risky integer operations during development and in builds so that incorrect data types are corrected early.

Coding Standard 2

Coding Standard	Label	Name of Standard Validate data values and ranges before use.
Data Value	STD-002-CPP	Data that comes from users, files, or other systems can be out of range, negative when it should be positive, or otherwise invalid. Using unvalidated values in array indexes, loop bounds, memory allocations, or calculations can cause crashes, buffer overflows, or logic errors. This standard requires developers to check that values fall within expected ranges and are reasonable before using them in security-sensitive operations.

Noncompliant Code

This example reads a size value from an external source and uses it directly to allocate an array and index into it without checking whether the value is reasonable. If count is very large or negative, the program can overflow memory or access out-of-bounds elements.

```
#include <iostream>

void process_items()
{
   int count;
   std::cout << "Enter number of items: ";
   std::cin >> count; // no validation

   int* items = new int[count]; // may allocate wrong size or fail

   for (int i = 0; i < count; ++i)
   {
      items[i] = i * 2; // may write out of bounds
   } delete[] items;
}
```

Compliant Code

This example validates the input value to ensure it is positive and below a defined maximum before using it in an allocation and loop. By enforcing a safe range, the code prevents negative sizes, overly large allocations, and out-of-bounds access.

```
#include <iostream>
#include <limits>
```

Compliant Code

```
void process_items_safe()
{
   int count;
   std::cout << "Enter number of items (1-1000): ";
   std::cin >> count;

   const int MAX_ITEMS = 1000;

   if (count <= 0 || count > MAX_ITEMS)
   {
      std::cout << "Invalid item count." << std::endl;
      return;
   }

   int* items = new int[count];

   for (int i = 0; i < count; ++i)
   {
      items[i] = i * 2;
   }
   delete[] items;
}
```

Principles(s): Principle 1: Validate Input Data. This standard directly supports validating all external numeric values (such as counts and sizes) before using them in allocations, loops, or indexing. Principle 8: Defense in Depth. Range checks on values add another layer of protection if earlier validation or input filtering misses an unsafe value.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Likely	Medium	High	2]

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Bounds and suspicious value usage checks	Detects questionable array sizes, loop bounds, and value usage that may indicate missing range validation.
SonarQube or SonarCloud C++ analyzer	Classroom version	C++ input-validation and untrusted data rules	Highlights use of untrusted values in allocations, loops, and calculations so developers can

Tool	Version	Checker	Description Tool
			add appropriate range checks.

Coding Standard 3

Coding Standard	Label	Name of Standard Use bounded string operations and safe input handling to prevent buffer overflows.
String Correctness	STD-003-CPP	Unbounded string operations that do not limit the number of characters written can overflow fixed-size buffers and corrupt memory. Functions that read or copy strings must always respect the destination buffer size and ensure that strings are properly null-terminated. This standard requires developers to use safer string handling patterns, such as std::string or explicit length checks, to avoid classic buffer overflow vulnerabilities.

Noncompliant Code

This example reads user input directly into a fixed-size char array without any limit on how many characters the user can type. If the user enters more than the buffer can hold, the extra characters overwrite adjacent memory and may crash the program or create a security vulnerability.

```
#include <iostream>

void read_username_noncompliant()
{
   char username[16]; // fixed-size buffer
   std::cout << "Enter username: ";
   std::cin >> username; // no length check
   std::cout << "Welcome, " << username << std::endl;
}
```

Compliant Code

This example reads the input into a std::string first and then copies at most sizeof(username) - 1 characters into the fixed-size buffer. It explicitly adds a null terminator, which prevents buffer overflow and ensures the string is properly terminated.

```
#include <iostream>
#include <string>

void read_username_compliant()
{
   char username[16]; // fixed-size buffer
   std::string temp;

   std::cout << "Enter username: ";
   std::cin >> temp;

   const std::size_t max_len = sizeof(username) - 1;
   std::size_t copy_len = (temp.size() < max_len) ? temp.size() :
```

Compliant Code

```
max_len;

   temp.copy(username, copy_len);
   username[copy_len] = '\0';

   std::cout << "Welcome, " << username << std::endl;
}
```

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Critical	Possible	Medium	Very High	1

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Out-of-bounds and buffer overrun checks	Detects string and array indexing that may exceed buffer bounds, including unsafe loops that ignore buffer size.
Visual Studio Code Analysis for C++	Current classroom version	Buffer overrun and range checking warnings	Warns when string or buffer operations may write past the end of an array or fail to terminate strings correctly.

Coding Standard 4

Coding Standard	Label	Name of Standard Use bounded string operations and safe input handling to prevent buffer overflows.
SQL Injection	STD-005-CPP	Manual memory management with raw new and delete can easily lead to leaks, double deletes, and dangling pointers. Using RAII (Resource Acquisition Is Initialization) and smart pointers such as std::unique_ptr and std::shared_ptr ties the lifetime of allocated memory to C++ objects. This standard reduces memory-related vulnerabilities by ensuring that resources are automatically released when they go out of scope, even when exceptions or early returns occur.

Noncompliant Code

This example allocates an object with new and may return early without calling delete. It also passes the raw pointer around without a clear owner, which makes it easy to leak memory or accidentally delete the same pointer twice.

```
#include <iostream>

struct Widget
{
   void doWork(){ std::cout << "Working\n"; }
};

void process_widget()
{
   Widget* w = new Widget(); // raw allocation

   if (!w)
   {
      return; // early return, potential leak
   }

   w->doWork();
   // developer might forget this delete in more complex code
   delete w;
}
```

Compliant Code

This example uses std::unique_ptr so that the Widget is automatically destroyed when it goes out of scope. There is no need to call delete manually, and early returns cannot leak memory because the smart pointer cleans up the object.

Compliant Code

```
#include <iostream>
#include <memory>

struct Widget
{
   void doWork() { std::cout << "Working\n"; }
};

void process_widget_safe()
{
   std::unique_ptr<Widget> w = std::make_unique<Widget>();

   if (!w)
   {
      return; // unique_ptr will still clean up correctly
   }
   w->doWork();
   // automatic cleanup when w goes out of scope
}
```

Principles(s): Principle 1: Validate Input Data. This standard requires validating and normalizing all user input before it reaches the database so that unexpected or malicious values are rejected early.
Principle 7: Sanitize Data Sent to Other Systems. The database is treated as an external system, and all data sent to it must be sanitized and passed as parameters instead of being concatenated into SQL strings.
Principle 8: Defense in Depth. In addition to parameterized queries, the application enforces least privilege accounts and careful error handling, so even if an injection attempt succeeds, its impact is limited.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Critical	Likely	Medium	Very High	1

Automation

Tool	Version	Checker	Description Tool
SonarQube or SonarCloud C++ analyzer	Classroom version	SQL injection rules (for example rules mapped to CWE-89)	Scans code for string concatenation patterns used to build SQL statements and flags places where untrusted input is inserted directly into query strings.

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Security checks for suspicious string building and external input usage	Helps identify functions that build SQL text using user input so they can be refactored to use parameterized queries.
Secure code review checklist	N/A	“No direct string concatenation in SQL commands”	During peer review, every database call is checked to ensure it uses prepared statements or stored procedures instead of concatenated SQL.

Coding Standard 5

Coding Standard	Label	Name of Standard Use RAII and smart pointers to manage dynamic memory safely.
Memory Protection	STD-005-CPP	Manual memory management with raw new and delete can easily lead to leaks, double deletes, and dangling pointers. Using RAII (Resource Acquisition Is Initialization) and smart pointers such as std::unique_ptr and std::shared_ptr ties the lifetime of allocated memory to C++ objects. This standard reduces memory-related vulnerabilities by ensuring that resources are automatically released when they go out of scope, even when exceptions or early returns occur.

Noncompliant Code

This example allocates an object with new and may return early without calling delete. It also passes the raw pointer around without a clear owner, which makes it easy to leak memory or accidentally delete the same pointer twice.

```
#include <iostream>

struct Widget
{
   void doWork() { std::cout << "Working\n"; }
};

   void process_widget()
{
   Widget* w = new Widget(); // raw allocation

   if (!w)
   {
      return; // early return, potential leak
   }
   w->doWork(); // developer might forget this delete in more complex
code
   delete w;
}
```

Compliant Code

This example uses std::unique_ptr so that the Widget is automatically destroyed when it goes out of scope. There is no need to call delete manually, and early returns cannot leak memory because the smart pointer cleans up the object.

Compliant Code

```
#include <iostream>
#include <memory>

struct Widget
{
   void doWork() { std::cout << "Working\n"; }
};

void process_widget_safe()
{
   std::unique_ptr<Widget> w = std::make_unique<Widget>();

   if (!w)
   {
      return; // unique_ptr will still clean up correctly
   }

   w->doWork();
   // automatic cleanup when w goes out of scope
}
```

Principles(s): Principle 6: Principle of Least Privilege. This standard limits direct access to raw memory by requiring clear ownership and minimizing the number of places that can allocate or free memory.
Principle 8: Defense in Depth. Using RAII objects and smart pointers provides an extra safety layer so that memory is still released correctly even if some error paths are missed.
Principle 10: Adopt a Secure Coding Standard. This standard follows secure C++ guidance to avoid manual new and delete when safer abstractions such as smart pointers and containers are available.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	High	2

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Memory leak, double free, and use-after-free checks	Analyzes code paths to find allocations that are not freed, frees that occur more than once, and pointers that may be used after the memory is

Tool	Version	Checker	Description Tool
			released.
AddressSanitizer (ASan) in test builds	Compiler-provided version	Heap use-after-free, buffer overflow, and leak detection	Runtime sanitizer used during automated tests to detect memory safety violations that static analysis might miss.

Coding Standard 6

Coding Standard	Label	Name of Standard Use assertions for internal assumptions, not for validating external input.
Assertions	STD-006-CPP	Assertions are meant to document and check assumptions that should always be true in correct code, such as invariants or internal state relationships. They can be disabled in release builds, so they must not be used to validate user input, file data, or any other external values. This standard helps ensure that security and correctness do not depend on assertions that may not run in production.

Noncompliant Code

This example uses assert to check that a user-supplied value is within range. If assertions are turned off in a release build, the check disappears and the function may use an invalid value without any protection.

```
#include <cassert>
#include <iostream>

void process_age_noncompliant(int age)
{
   // age comes from user input
   assert(age >= 0 && age <= 120); // not reliable for input validation

   std::cout << "Processing age: " << age << std::endl;
}
```

Compliant Code

This example uses an explicit runtime check to validate the user-supplied value and reserves assertions for internal invariants. The input is safely rejected if it is out of range, even when assertions are disabled in production builds.

```
#include <cassert>
#include <iostream>

void process_age_compliant(int age)
{
   // Runtime check for external input
   if (age < 0 || age > 120)
   {
      std::cout << "Invalid age value." << std::endl;
      return;
   }
```

Compliant Code

```
    // Assertion for an internal invariant
    assert(age >= 0 && age <= 120);

    std::cout << "Processing age: " << age << std::endl;
}
```

Principles(s): Principle 3: Architect and Design for Security Policies. This standard requires assertions to express internal invariants and programmer assumptions, not to enforce runtime security checks that users can disable in release builds.
Principle 8: Defense in Depth. Properly used assertions catch logic errors during development so they can be fixed before they become exploitable vulnerabilities in production.
Principle 10: Adopt a Secure Coding Standard. This standard follows secure coding guidelines that forbid using assertions for input validation, access control, or other security decisions.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Possible	Low	Medium	3

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Suspicious assert expressions and constant conditions	Detects assertions that use assignments, always-true conditions, or expressions that look like they were meant as comparisons, indicating that the assert is not enforcing the intended constraint.
SonarQube or SonarCloud C++ analyzer	Classroom version	Misuse of assertions and “assert used for argument checking” rules	Flags code that relies on assertions for parameter validation or security checks so those checks can be moved into normal runtime logic.

Coding Standard 7

Coding Standard	Label	Name of Standard Use exceptions for error handling and maintain strong exception safety.
Exceptions	STD-007-CPP	Exceptions allow errors to be signaled without mixing error codes into normal return values, but they must be used carefully. Functions should either complete successfully or leave program state unchanged if an exception is thrown, and resources must still be released. This standard helps prevent inconsistent state, resource leaks, and security issues that can occur when code is not exception-safe.

Noncompliant Code

This example allocates memory with new, modifies shared state, and then may throw an exception without cleaning up the resource. If an exception occurs after the allocation, the function leaks memory and may leave the global state partially updated.

```
#include <iostream>
#include <string>

std::string globalName;

void set_name_noncompliant(const std::string& input)
{
   char* buffer = new char[input.size() + 1]; // raw allocation

   // Copy may throw (for example, if memory is low)
   for (std::size_t i = 0; i < input.size(); ++i)
   {
      buffer[i] = input[i];
   }
   buffer[input.size()] = '\0';

   // If an exception happened above, this line is never reached
   globalName = buffer;
   delete[] buffer; // may never be called
}
```

Compliant Code

This example uses std::string and RAII, so all resources are managed automatically and no manual new or delete is needed. If an exception is thrown during the operation, the C++ runtime cleans up temporary objects,

Compliant Code

and either the function completes successfully or the program state remains unchanged.

```
#include <iostream>
#include <string>

std::string globalNameSafe;

void set_name_compliant(const std::string& input)
{
    // std::string manages its own memory and is exception-safe
    std::string temp = input; // may throw, but no leaks

    // Commit the change only after all operations succeed
    globalNameSafe = temp;
}
```

Principles(s): Principle 3: Architect and Design for Security Policies. This standard requires a clear strategy for how exceptions are thrown, propagated, and handled so that error paths do not bypass security checks or leave the system in an inconsistent state.
Principle 8: Defense in Depth. Well-designed exception handling adds a layer of resilience by ensuring that resources are cleaned up and partial failures do not leave behind exploitable conditions.
Principle 9: Apply Quality Assurance Techniques. Consistent exception usage, combined with unit tests and negative tests, helps verify that error conditions are handled correctly and do not expose sensitive information.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	High	2

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Exception safety and catch-all usage checks	Flags empty catch blocks, overly broad catch-all handlers that hide errors, and functions that throw in places marked as nothrow.
SonarQube or SonarCloud C++ analyzer	Classroom version	Rules for exception handling and error hiding (for example, empty catch blocks, ignored return values, and logging of sensitive details)	Highlights exception handling patterns that swallow errors or expose internal details so developers can replace them with structured error handling

Tool	Version	Checker	Description Tool
			and safe logging.

Coding Standard 8

Coding Standard	Label	Name of Standard Log security-relevant events without exposing sensitive data.
Secure Logging	STD-008-CPP	Logging is critical for detecting attacks, troubleshooting issues, and supporting incident response, but logs can also leak sensitive information if they are not handled carefully. Security-relevant events such as authentication failures, access denials, and unexpected errors should be recorded, but secrets like passwords, full credit card numbers, or personal identifiers must never be written to logs. This standard helps ensure logs are useful for security monitoring while still protecting confidentiality and complying with privacy requirements.

Noncompliant Code

This example logs the full username and password when a login fails. If an attacker gains access to the log files, they can see users' passwords in clear text, which is a serious confidentiality risk.

```
#include <iostream>
#include <string>

void log_failed_login_noncompliant(const std::string& username, const
std::string& password)
{
   std::cerr << "Login failed for user: " << username << " with password:
" << password << std::endl;
}
```

Compliant Code

This example logs that a login failed and includes only a generic reason and the username, without recording the password or other secrets. The log is still useful for detecting repeated failures or brute-force attempts, but it does not expose sensitive data.

```
#include <iostream>
#include <string>

void log_failed_login_compliant(const std::string& username)
{
   std::cerr << "Login failed for user: " << username << " (invalid
credentials)" << std::endl;
}
```

Principles(s): Principle 3: Architect and Design for Security Policies. This standard requires planning what to log, how long to keep it, and who can see it so that logs support security monitoring without violating privacy or leaking secrets.
Principle 6: Principle of Least Privilege. Access to logs is limited to users and services that genuinely need them, and sensitive details are masked or removed so that even authorized viewers only see what they require.
Principle 8: Defense in Depth. Secure logging adds a layer of detection and traceability: even if an attacker reaches the system, well-designed logs help identify misuse and support incident response.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	High	2

Automation

Tool	Version	Checker	Description Tool
Static code analyzer (for example, SonarQube / SonarCloud C++ rules)	Classroom version	Logging and hardcoded credentials rules	Detects log statements that include variables likely to contain credentials or personal data, and flags hardcoded secrets that should never be logged.
Log management platform with pattern-based filters (for example, a central log aggregator)	Current platform version	Redaction and PII pattern filters	Automatically scans outgoing logs for sensitive patterns such as credit card numbers, access tokens, and government IDs and masks or drops those fields before storage or forwarding.

Coding Standard 9

Coding Standard	Label	Name of Standard Return generic error messages to users and keep detailed information in logs only.
Coding standards an	STD-009-CPP	Detailed error messages that expose stack traces, SQL queries, file paths, or configuration details can give attackers valuable clues about how the system works. Users still need to know that something went wrong and what they can do next, but internal details should be written to secure logs instead of being shown on the screen. This standard helps prevent information disclosure while still supporting troubleshooting and incident response.

Noncompliant Code

This example directly prints the full exception message, including internal details, to the user. If the exception contains file paths, SQL statements, or other sensitive information, an attacker can use that data to plan further attacks.

```
#include <iostream>
#include <stdexcept>

void save_data_noncompliant()
{
  try
  {
    // ... code that may throw ...
    throw std::runtime_error("Database error in save_data at /srv/app/db.cpp:42");
  }
  catch (const std::exception& ex)
  {
    // Exposes internal details to the user
    std::cout << "Error: " << ex.what() << std::endl;
  }
}
```

Compliant Code

This example logs the detailed error message for developers and returns a simple, user-friendly message to the user. The user learns that the operation failed and what to do next, but no internal paths or configuration details are revealed.

```
#include <iostream>
#include <fstream>
#include <stdexcept>

void log_error(const std::string& details)
```

Compliant Code

```
{
   std::ofstream log("error.log", std::ios::app);
   log << details << std::endl;
}
void save_data_compliant()
{
   try
   {
      // ... code that may throw ...
      throw std::runtime_error("Database error in save_data at
/srv/app/db.cpp:42");
   }
   catch (const std::exception& ex)
   {
      // Log internal details for developers
      log_error(ex.what());

      // Show a generic message to the user
      std::cout << "Sorry, something went wrong while saving your data. "
"Please try again later." << std::endl;
   }
}
```

Principles(s): Principle 3: Architect and Design for Security Policies. This standard enforces a clear policy that separates internal diagnostic information from user-visible error messages so the system fails safely without exposing implementation details.
Principle 7: Sanitize Data Sent to Other Systems. Error messages sent back to users are treated as external output and are sanitized so they do not include file paths, SQL statements, or configuration values.
Principle 8: Defense in Depth. Even if an internal error occurs, controlled error messages prevent attackers from learning how to exploit deeper vulnerabilities while still allowing secure logs to capture what happened.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Low	High	2

Automation

Tool	Version	Checker	Description Tool
SonarQube or SonarCloud C++ analyzer	Classroom version	Error handling and logging rules (avoid exposing internal details)	Flags places where exception messages, stack traces, or raw SQL and file paths are written directly to standard output or

Tool	Version	Checker	Description Tool
			returned to users, so they can be replaced with generic messages and secure logging.
Secure code review checklist	N/A	“No internal details in user-visible error messages”	During peer review, all error messages are checked to ensure only high-level, non-sensitive text is shown to users and detailed information is recorded only in protected logs.

Coding Standard 10

Coding Standard	Label	Name of Standard Validate all external input at API and UI boundaries using whitelists and range checks.
Input Validation for User Interfaces / APIs	STD-010-CPP	Any data coming from user interfaces, web requests, files, or other services must be treated as untrusted and validated before it is used. Robust input validation at API and UI boundaries checks type, length, format, and allowed ranges, rejecting values that do not match expected patterns. This standard helps prevent injection attacks, buffer overflows, and data corruption by ensuring only well-formed, safe data enters the system.

Noncompliant Code

This example takes a string from an API or UI and converts it directly to an integer without any checks. If the caller sends a non-numeric or out-of-range value, the conversion can fail or produce unexpected results that may be used in array indexes or allocations.

```
#include <iostream>
#include <string>
#include <cstdlib>

void handle_request_noncompliant(const std::string& itemCountParam)
{
   // No validation of input format or range
   int count = std::atoi(itemCountParam.c_str());


   int* items = new int[count]; // may allocate wrong size or fail
   for (int i = 0; i < count; ++i)
   {
      items[i] = i;
   }
   delete[] items;
}
```

Compliant Code

This example validates that the input is numeric and within a safe range before converting it and using it in an allocation. By enforcing type, format, and range checks at the API boundary, the function rejects bad input early and avoids unsafe memory operations.

Compliant Code

```
#include <iostream>
#include <string>
#include <limits>

bool parse_positive_int(const std::string& text, int& value)
{
   if (text.empty())
      return false;

   int result = 0;
   for (char c : text)
   {
      if (c < '0' || c > '9')
         return false;
      int digit = c - '0';
      if (result > (std::numeric_limits<int>::max() - digit) / 10)
         return false; // overflow
      result = result * 10 + digit;
   }

   value = result;
   return true;
}
void handle_request_compliant(const std::string& itemCountParam)
{
   const int MAX_ITEMS = 1000;
   int count = 0;

   if (!parse_positive_int(itemCountParam, count) || count <= 0 || count
> MAX_ITEMS)
   {
      std::cout << "Invalid item count." << std::endl; return;
   }

   int* items = new int[count];

   for (int i = 0; i < count; ++i)
   {
      items[i] = i;
   }
   delete[] items;
}
```

Principles(s): Principle 1: Validate Input Data. This standard is a direct application of validating all untrusted input at UI and API boundaries for type, length, format, and range before it is used anywhere in the system.
Principle 8: Defense in Depth. Early validation at the boundary adds a protection layer even if later checks fail, reducing the chance that malformed or hostile data reaches deeper components.
Principle 10: Adopt a Secure Coding Standard. This standard follows secure coding guidance from CERT and OWASP that require strict validation of all external data to prevent injection, overflow, and corruption.

Threat Level

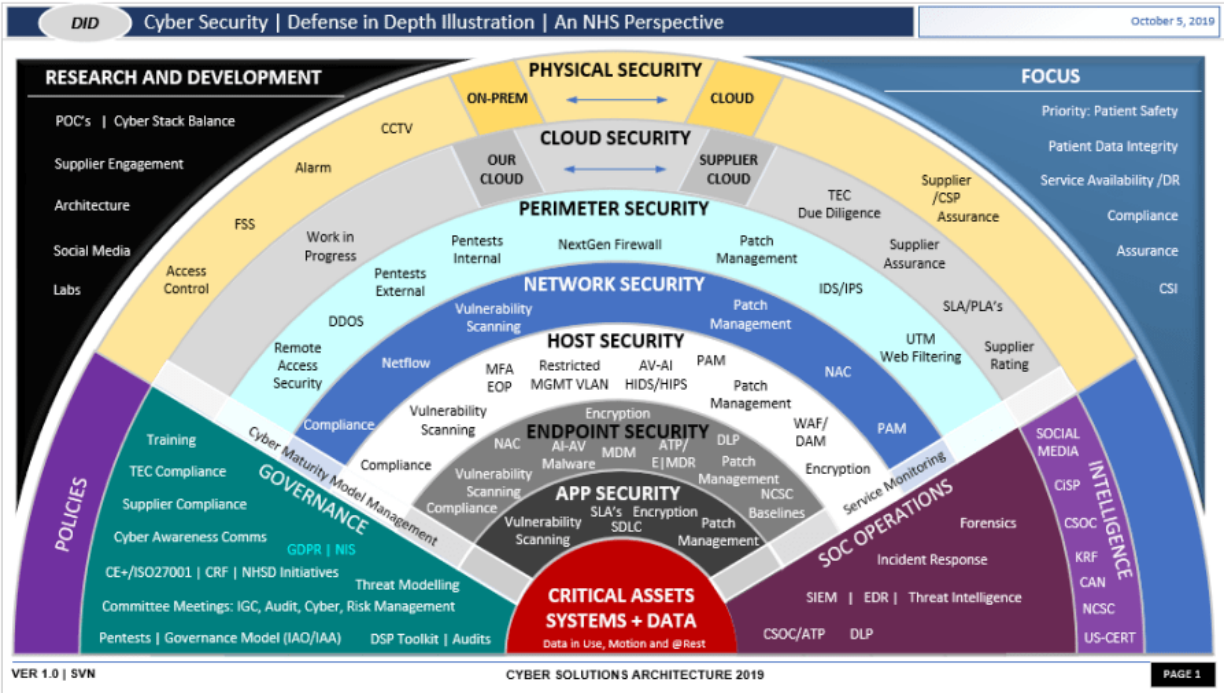
Severity	Likelihood	Remediation Cost	Priority	Level
Critical	Likely	Medium	High	1

Automation

Tool	Version	Checker	Description Tool
Cppcheck	2.19.0	Bounds, uninitialized values, and suspicious value usage checks	Highlights use of untrusted parameters in allocations, loops, and array indexes without prior validation, which helps enforce this standard at the code level.
SonarQube or SonarCloud C++ analyze	Classroom version	Input validation and untrusted data rules	Detects functions that accept external input and use it directly in critical operations, prompting developers to add centralized validation routines like parse_positive_int.

Defense-in-Depth Illustration

This illustration provides a visual representation of the defense-in-depth best practice of layered security.



Project One

There are seven steps outlined below that align with the elements you will be graded on in the accompanying rubric. When you complete these steps, you will have finished the security policy.

Revise the C/C++ Standards

You completed one of these tables for each of your standards in the Module Three milestone. In Project One, add revisions to improve the explanation and examples as needed. Add rows to accommodate additional examples of compliant and noncompliant code. Coding standards begin on the security policy.

Risk Assessment

Complete this section on the coding standards tables. Enter high, medium, or low for each of the headers, then rate it overall using a scale from 1 to 5, 5 being the greatest threat. You will address each of the seven policy standards. Fill in the columns of severity, likelihood, remediation cost, priority, and level using the values provided in the appendix.

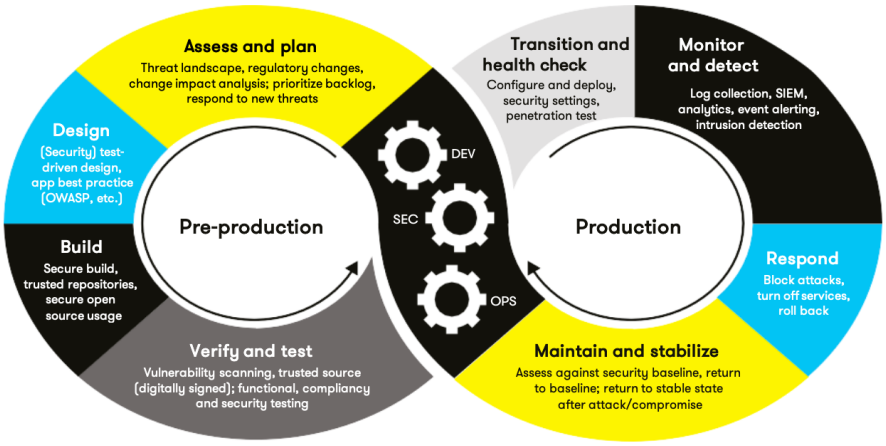
Automated Detection

Complete this section of each table on the coding standards to show the tools that may be used to detect issues. Provide the tool name, version, checker, and description. List one or more tools that can automatically detect this issue and its version number, name of the rule or check (preferably with link), and any relevant comments or description—if any. This table ties to a specific C++ coding standard.

Automation

Provide a written explanation using the image provided.

Green Pace



Automation will be used for the enforcement of and compliance to the standards defined in this policy. Green Pace already has a well-established DevOps process and infrastructure. Define guidance on where and how to modify the existing DevOps process to automate enforcement of the standards in this policy. Use the DevSecOps diagram and provide an explanation using that diagram as context.

Automation will be integrated into Green Pace's existing DevOps pipeline so that security checks run consistently every time code is built, tested, and deployed. In the Plan and Code stages, developers must follow the C/C++ coding standards in this policy and run static analysis tools such as Cppcheck and SonarQube locally to detect issues early. New features and bug fixes must include tests that cover secure behaviors, such as input validation, error handling, and correct use of memory and exceptions.

In the Build and Test stages, the continuous integration (CI) system automatically compiles the code, runs all unit and integration tests, and executes static analysis tools with the rules that enforce these standards. Builds will fail if critical or high-severity violations are found or if required tests do not pass, which prevents insecure code from being merged or released. This moves enforcement from manual reviews into automated, repeatable checks.

In the Release and Deploy stages, a security gate is required before deploying to any Green Pace environment. The gate includes a review of static analysis results, confirmation that tests have passed, and verification that the coding and system standards in this policy are being followed. Automated deployment pipelines must reject releases that do not meet the defined security criteria so that exceptions are intentional and documented, not accidental.

During Operate and Monitor, log aggregation, monitoring tools, and existing operating-system, firewall, and anti-malware logs are used to detect policy violations and suspicious behavior. Findings from production are fed back into the development backlog so teams can add new tests, refine coding standards, or enhance tools. This creates a DevSecOps feedback loop where automation continuously enforces standards and improves Green Pace's security posture over time.

Summary of Risk Assessments

Consolidate all risk assessments into one table including both coding and systems standards, ordered by standard number.

Rule	Severity	Likelihood	Remediation Cost	Priority	Level

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-001-CPP	High	Unlikely	Medium	High	2
STD-002-CPP	High	Likely	Medium	High	2
STD-003-CPP	Critical	Possible	Medium	Very High	1
STD-004-CPP	Critical	Likely	Medium	Very High	1
STD-005-CPP	High	Possible	Medium	High	2
STD-006-CPP	Medium	Possible	Low	Medium	3
STD-007-CPP	High	Possible	Medium	High	2
STD-008-CPP	High	Possible	Medium	High	2
STD-009-CPP	High	Possible	Low	High	2
STD-010-CPP	Critical	Likely	Medium	High	1

Create Policies for Encryption and Triple A

Include all three types of encryption (in flight, at rest, and in use) and each of the three elements of the Triple-A framework using the tables provided**.**

a. Explain each type of encryption, how it is used, and why and when the policy applies.
b. Explain each type of Triple-A framework strategy, how it is used, and why and when the policy applies.

Write policies for each and explain what it is, how it should be applied in practice, and why it should be used.

a. Encryption	**Explain what it is and how and why the policy applies.**
Encryption at rest	Encryption at rest protects data stored on physical media such as database files, application servers, local disks, and backups. At Green Pace, any storage that contains customer data, financial data, or other sensitive internal information must use strong encryption (for example, AES-based full-disk or database encryption) with keys managed in a secure key management system. This policy applies to on-premises servers, cloud storage services, and backup media so that if storage devices are lost, stolen, or improperly accessed, the data cannot be read without the encryption keys.

a. Encryption	Explain what it is and how and why the policy applies.
Encryption in flight	Encryption in flight protects data as it moves across networks between clients, services, and databases. At Green Pace, all external and internal application traffic that carries sensitive or authenticated data must use secure protocols such as HTTPS/TLS or TLS-secured database connections, and insecure protocols (such as plain HTTP or unencrypted database connections) must not be used for production systems. This policy applies to web APIs, browser-based clients, service-to-service calls, and administrative access so that attackers who intercept network traffic cannot read or modify the data.
Encryption in use	Encryption in use focuses on protecting sensitive data while it is actively being processed in memory or displayed to users. At Green Pace, applications must minimize how long sensitive data such as passwords, encryption keys, and personal information remain in memory, avoid writing such data to temporary files or logs, and restrict which processes and users can access it. This policy applies to in-memory caches, session data, and any feature that handles secrets, helping reduce the impact of memory-scraping malware, shoulder surfing, or unauthorized screen and process access.

b. Triple-A Framework*	Explain what it is and how and why the policy applies.
Authentication	Authentication verifies that a user or system is really who they claim to be before granting access to Green Pace resources. All applications must require unique user accounts and secure login mechanisms, such as strong passwords and, where possible, multi-factor authentication. This policy applies to any system that allows user logins, and all login attempts—both successful and failed—must be logged so that attempted misuse can be detected and investigated.
Authorization	Authorization controls what an authenticated user is allowed to do based on their role and job responsibilities. Green Pace will implement role-based access control for applications, with least-privilege access to databases, files, and administrative features, and any change to a user's level of access must follow an approved request process. This policy applies when adding new users, changing roles, or granting access to sensitive data, and all permission changes must be logged so there is a clear history of who can do what and why.
Accounting	Accounting (or auditing) tracks what users and systems do after they have been authenticated and authorized. All systems governed by this policy must record key events, including user logins, file access, database changes, and creation or removal of user accounts, and those logs must be protected from tampering and retained for an appropriate period. These records provide evidence of

b. Triple-A Framework*	Explain what it is and how and why the policy applies.
	compliance with this policy and support investigations and reviews by the risk management committee and the OCISO.

*****Use this checklist for the Triple A to be sure you include these elements in your policy:

- User logins
- Changes to the database
- Addition of new users
- User level of access
- Files accessed by users

Map the Principles

Map the principles to each of the standards, and provide a justification for the connection between the two. In the Module Three milestone, you added definitions for each of the 10 principles provided. Now it's time to connect the standards to principles to show how they are supported by principles. You may have more than one principle for each standard, and the principles may be used more than once. Principles are numbered 1 through 10. You will list the number or numbers that apply to each standard, then explain how each of these principles supports the standard. This exercise demonstrates that you have based your security policy on widely accepted principles. Linking principles to standards is a best practice.

NOTE: Green Pace has already successfully implemented the following:

- Operating system logs
- Firewall logs
- Anti-malware logs

The only item you must complete beyond this point is the Policy Version History table.

Audit Controls and Management

Every software development effort must be able to provide evidence of compliance for each software deployed into any Green Pace managed environment.

Evidence will include the following:

- Code compliance to standards
- Well-documented access-control strategies, with sampled evidence of compliance
- Well-documented data-control standards defining the expected security posture of data at rest, in flight, and in use
- Historical evidence of sustained practice (emails, logs, audits, meeting notes)

Enforcement

The office of the chief information security officer (OCISO) will enforce awareness and compliance of this policy, producing reports for the risk management committee (RMC) to review monthly. Every system deployed in any environment operated by Green Pace is expected to be in compliance with this policy at all times.

Staff members, consultants, or employees found in violation of this policy will be subject to disciplinary action, up to and including termination.

Exceptions Process

Any exception to the standards in this policy must be requested in writing with the following information:

- Business or technical rationale
- Risk impact analysis
- Risk mitigation analysis
- Plan to come into compliance
- Date for when the plan to come into compliance will be completed

Approval for any exception must be granted by chief information officer (CIO) and the chief information security officer (CISO) or their appointed delegates of officer level.

Exceptions will remain on file with the office of the CISO, which will administer and govern compliance.

Distribution

This policy is to be distributed to all Green Pace IT staff annually. All IT staff will need to certify acceptance and awareness of this policy annually.

Policy Change Control

This policy will be automatically reviewed annually, no later than 365 days from the last revision date. Further, it will be reviewed in response to regulatory or compliance changes, and on demand as determined by the OCISO.

Policy Version History

Version	Date	Description	Edited By	Approved By
1.0	08/05/2020	Initial Template	David Buksbaum	
1.1	06/08/2026	Initial Green Pace secure development policy for CS-405 Project One	Gary Travis	Olga Mill

Appendix A Lookups

Approved C/C++ Language Acronyms

Language	Acronym
C++	CPP
C	CLG
Java	JAV